

OpenDccCLib — Architecture (As-Built)

Generated 2026-09-25 from the repository sources by documentation/guides/build_pdfs.py

Table of Contents

1. Overview	3
2. Feature flags	4
3. User-configurable constants	5
4. The <code>dcc_config_t</code> struct	6
5. Public API surface	7
Lifecycle & ISR entry points (<code>dcc_config.h</code>)	7
Application modules (role-first naming)	7
6. Internal modules	8
7. Execution contexts	10
8. Key invariants	11

Verified against commit a451be7 (2026-09-23). This document describes the library as it is implemented today. For the original design intent (which predates the role-first naming refactor), see [archive/OpenDccCLib_Requirements.md](#). For what is done vs. pending, see [ComplianceOverview.md](#).

1. Overview

OpenDccCLib is a portable C library implementing NMRA DCC (S-9.1 through S-9.3.2) for both command-station and decoder roles. Design constraints:

- **No dynamic memory.** All buffers and state are statically sized by `USER_DEFINED_DCC_*` constants.
- **No OS / RTOS.** Runs from a main loop plus a small number of ISRs.
- **Dependency injection everywhere.** Cross-module calls go through `interface_dcc_*_t` function-pointer structs, never direct includes. Only `dcc_config.c` (the wiring module) includes every module header.
- **Portable hardware layer.** The library never touches hardware directly; the user supplies driver callbacks in one `dcc_config_t` struct.

The user includes a single header — `dcc_config.h` — populates one struct, and calls `DccConfig_initialize()`.

2. Feature flags

Defined by the user in `dcc_user_config.h`, validated with `#error/#warning` in `dcc_config.h`.

Flag	Enables
<code>DCC_COMPILE_COMMAND_STATION</code>	Packet encoding, bit framing, scheduler, service mode, RailCom cutout + receive
<code>DCC_COMPILE_DECODER</code>	Packet decoding, CV storage, RailCom response encoding
<code>DCC_COMPILE_ACCESSORY_DECODER</code>	Accessory-decoder RailCom (SRQ, status, time, error)
<code>DCC_COMPILE_SERVICE_MODE_DIRECT</code>	Direct byte/bit programming (requires CS)
<code>DCC_COMPILE_SERVICE_MODE_PAGED</code>	Paged programming (requires CS)
<code>DCC_COMPILE_SERVICE_MODE_REGISTER</code>	Physical register programming (requires CS)
<code>DCC_COMPILE_SERVICE_MODE_ADDRESS</code>	Address-only programming (requires CS)

Optional hardware is gated at runtime, not compile time. RailCom and service mode compile in with their parent role; if the hardware isn't present, the user leaves the corresponding callback NULL and the feature self-disables. The four service-mode flags each require `DCC_COMPILE_COMMAND_STATION`. A device may define both CS and DECODER for booster/repeater use.

3. User-configurable constants

Set in `dcc_user_config.h`:

- `USER_DEFINED_DCC_SCHEDULER_SLOT_COUNT` — max concurrent active scheduler slots
- `USER_DEFINED_DCC_MAX_LOCOS` — locomotives tracked by the application's loco table
- `USER_DEFINED_DCC_PREAMBLE_BITS_OPS` — operations-mode preamble (≥ 14 ; ≥ 16 with RailCom)
- `USER_DEFINED_DCC_RAILCOM_BUFFER_DEPTH` — RailCom receive ring-buffer depth
- `USER_DEFINED_DCC_SERVICE_MODE_RETRIES`
- `USER_DEFINED_DCC_ACK_THRESHOLD_MA`, `..._ACK_MIN_DURATION_US`, `..._ACK_MAX_DURATION_US`, `..._ACK_DROPOUT_TOLERANCE_US`
- `USER_DEFINED_DCC_DECODER_MAX_FUNCTIONS`, `..._DECODER_PACKET_QUEUE_DEPTH` (≥ 2 , one slot reserved)

One-shot packet repeat counts are not user constants: the builders set them from the `DCC_REPEAT_*` table in `dcc_defines.h` (CV write 2, verify 1, date 3, time 1, accessory NOP/stop 1, everything else 2); an application may overwrite `repeat_count` after a builder returns, and 0 means the scheduler never sends the packet.

4. The `dcc_config_t` struct

One user-facing struct (`dcc_config.h`). Required fields must be non-NULL; NULL-optional fields disable the associated feature at runtime.

Common (always required): `lock_shared_resources`, `unlock_shared_resources`, `get_timestamp_usec`.

Command Station: a shared fixed-period DCC timer (`shared_timer_start/stop`, clocked at `DCC_ONE_BIT_HALF_PERIOD_US` = 58 μ s) plus a RailCom one-shot timer (`railcom_timer_start/stop`). Per-channel hardware is described by `dcc_output_hw_t` for `main_track` and `service_track` (each with `timer_start/stop`, `pin_toggle`, `track_power_set`, optional `current_sense_read`, and an optional nested `dcc_railcom_hw_t`), plus the five RailCom cutout periods (0 = spec default). Optional callbacks: `on_packet_sent`, `on_accessory_srq`, and `on_railcom_datagram_result` inside `dcc_railcom_hw_t`. Service-mode results are delivered per call through the task callbacks (`on_complete`, `on_progress`, `on_detect`), not through a config callback.

Decoder: required `cv_read/cv_write/cv29_apply_supported_features`; NULL-optional `factory_reset`, `cv_read_indexed/cv_write_indexed`, `railcom_tx_pin_set` + `railcom_delay_us` + `on_railcom_request`, `start_ack_pulse/stop_ack_pulse`; and a set of `on*_command` notification callbacks plus `on_failsafe_entered/exited`. The DCC edge interrupt is masked during a RailCom transmit through `lock_shared_resources`; there is no separate edge-IRQ hook. Note the CV callbacks (`on_cv_write_command`, `on_cv_verify_command`, `on_cv_bit_command`) carry a `bool service_mode` argument so the application can distinguish service-mode programming from POM.

5. Public API surface

Lifecycle & ISR entry points (`dcc_config.h`)

- `DccConfig_initialize(const dcc_config_t *)`
- `DccConfig_run()` — main loop; **all application callbacks fire from here**
- `DccConfig_58us_timer_isr()` — [CS] shared bit timer
- `DccConfig_railcom_oneshot_timer_isr()` — [CS] RailCom cutout state machine
- `DccConfig_set_railcom_cutout_timing()`, `DccConfig_cancel_railcom_cutout()`, `DccConfig_railcom_cutout_is_active()` — [CS] runtime cutout control
- `DccConfig_100ms_timer_tick()` — [CS] timeouts / housekeeping
- `DccConfig_decoder_edge_isr(uint32_t timestamp_usec)` — [DECODER] input-capture edge

Application modules (role-first naming)

Module	Role	Public prefix
<code>dcc_application_command_station_main_track</code>	CS	<code>DccApplicationCommandStationMainTrack_</code> — <code>power_on/off</code> , <code>send_packet</code> , <code>add_to_auto_refresh</code> , <code>remove_from_auto_refresh</code> , <code>remove_all_auto_refresh</code>
<code>dcc_application_command_station_service_track</code>	CS	<code>DccApplicationCommandStationServiceTrack_</code> — <code>power_on/off</code> , <code>enter/exit_service_mode</code> , <code>is_service_mode_active</code> , <code>direct/paged/register/address write/verify</code>
<code>dcc_application_command_station_packet</code>	CS	<code>DccApplicationCommandStationPacket_load_*</code> — packet builders
<code>dcc_application_decoder_cv</code>	DECODER	<code>DccApplicationDecoderCv_</code> — <code>read</code> , <code>write</code> , <code>is_locked</code>
<code>dcc_application_decoder_railcom</code>	DECODER	<code>DccApplicationDecoderRailcom_send_*</code> — address feedback, POM response, dynamic data, <code>ack/nack</code> , track search, <code>cv auto transfer</code> , <code>raw</code>
<code>dcc_application_accessory_decoder_railcom</code>	ACCESSORY	<code>DccApplicationAccessoryDecoderRailcom_</code> — <code>SRQ</code> , status (1/4/extended), time/error report, cutout/stop hooks

Migration note: the pre-refactor modules `dcc_application_main_track` and `dcc_application_service_track` still exist and are still compiled/tested alongside the role-first modules above. Retiring them is tracked in *ComplianceOverview.md*. The third one, `dcc_packet_encoder`, was removed on 2026-09-23: it was an uncalled duplicate of `dcc_application_command_station_packet` and had drifted (it still carried the `repeat_count = 0` "never transmitted" defect fixed in the live module).

6. Internal modules

Module	Role	Responsibility
dcc_config	always	Wiring: builds every interface struct, owns all module contexts, initialize()/run()/ISR dispatch
dcc_types	always	Typedefs, user-constant validation
dcc_definitions	always	Protocol constants: timing, instruction masks, CV numbers, RailCom IDs
dcc_scheduler	CS	Priority queue, duplicate combining, paced auto-refresh: a changed slot is sent DCC_REFRESH_PROMPT_SENDS times at full rate, then kept alive every DCC_REFRESH_COLD_CYCLES packet cycles and never later than DCC_REFRESH_COLD_MAX_CYCLES (selection order overdue, in-burst, due; DCC_REFRESH_COLD_CYCLES = 0 is the flat round-robin ring); one-shots are sent repeat_count times (the builders set the DCC_REPEAT_* defaults, 0 = never sent)
dcc_bit_encoder	CS	ISR bit framing from the shared fixed-period timer
dcc_railcom_cutout	CS	RailCom cutout timer state machine
dcc_railcom_command_station	CS	Receive drain after each cutout, Ch1/Ch2 datagram assembly, receive ring, address tagging
dcc_railcom_utilities	CS + DECODER	4/8 code words (S-9.3.2 Table 2): encode for the decoder roles, decode for the command station
dcc_service_mode_common	CS	Shared ACK detection, reset sequencing, retry
dcc_service_mode_{direct,paged,register,address}	CS	Per-mode programming primitives
dcc_service_mode_task_{direct,paged,register,address,detect}	CS	Read/write/verify orchestration on the primitives; mode detection
dcc_bit_decoder	DECODER	Edge-timestamp → bit classification → byte assembly
dcc_packet_decoder	DECODER	Parse bytes → structured commands, XOR, address match, deferred dispatch queue
dcc_cv_storage	DECODER	CV abstraction, decoder lock, factory reset, indexed CVs, CV29 feature mask
dcc_failafe	DECODER	S-9.2.4 packet time-out (CV11 in 100 ms units)

Module	Role	Responsibility
dcc_railcom_decoder	DECODER	RailCom transmit engine (bit-bang) and reply arming

Each module owns an `interface_dcc_<module>_t` of function pointers, populated by `dcc_config.c`. This makes every dependency mockable in unit tests and lets an MCU swap touch only the config struct.

7. Execution contexts

Three contexts, with a deliberately small locked region:

- **ISR (timer / edge):** bit encoder advances the bit state machine and toggles the output pin; RailCom cutout timing fires here; decoder edge classification happens here. ISR work is kept minimal. The decoder bit-bangs RailCom Tx from ISR context at 4 μ s intervals during the cutout (the CPU is otherwise idle then).
- **Main loop (`DccConfig_run()`):** scheduler selection, packet build, service-mode state machine, RailCom receive drain, decoded-packet dispatch, fail-safe timing, 6 ms ACK-pulse timing. **Every application callback fires from this context, never from an ISR.**
- **User API (any thread):** the `DccApplication*` calls write into shared state; on threaded hosts they must be protected by the user's `lock/unlock`.

The handoff between the ISR-level bit encoder and the main-loop scheduler is single-buffered: the encoder holds one active packet and a `packet_loaded` flag; the ISR clears the flag at the end bit and the main loop loads the next packet under `lock_shared_resources/unlock_shared_resources`. The cutout arm is deferred one tick so it lands on the end bit's last edge, because the encoder's state machine runs a half-bit ahead of the wire.

8. Key invariants

1. No direct cross-module includes in the core library — only via interface structs.
2. Every new function/type is wrapped in the appropriate `DCC_COMPILE_*` guard.
3. No dynamic memory; all sizing comes from `USER_DEFINED_DCC_*` constants.
4. CV numbers are 1-based everywhere; the 0-based wire encoding is confined to the packet builders (`dcc_application_command_station_packet.c`).
5. Callbacks are main-loop only; ISRs do pin/flag work and defer notification.